
OUTILS JAVA

Master I I.F.R.I

KOUKPAKI Jean-Pierre
Ingénieur informatique / Expert logiciel

SOMMAIRE

1. Introduction au langage Java
2. Les bonnes pratiques de développement en Java
3. Les design patterns
4. Gestion des dépendances projet avec Maven
5. Le framework Spring (Spring web, SpringData, SpringSecurity, SpringBoot...)
6. Développement d'interfaces graphiques avec les JSP
7. Evaluations pratiques

Introduction

Le langage JAVA

Langage de programmation orienté objet

Proche du langage C

Mis au point en 1990 par la société SUN Microsystems

Première présentation en 1995

Dernière version : Java 23 disponible depuis le 17 Septembre 2024

Java 23 fait suite à la LTS Java 21 et précède la prochaine LTS Java 25 prévue pour Septembre 2025

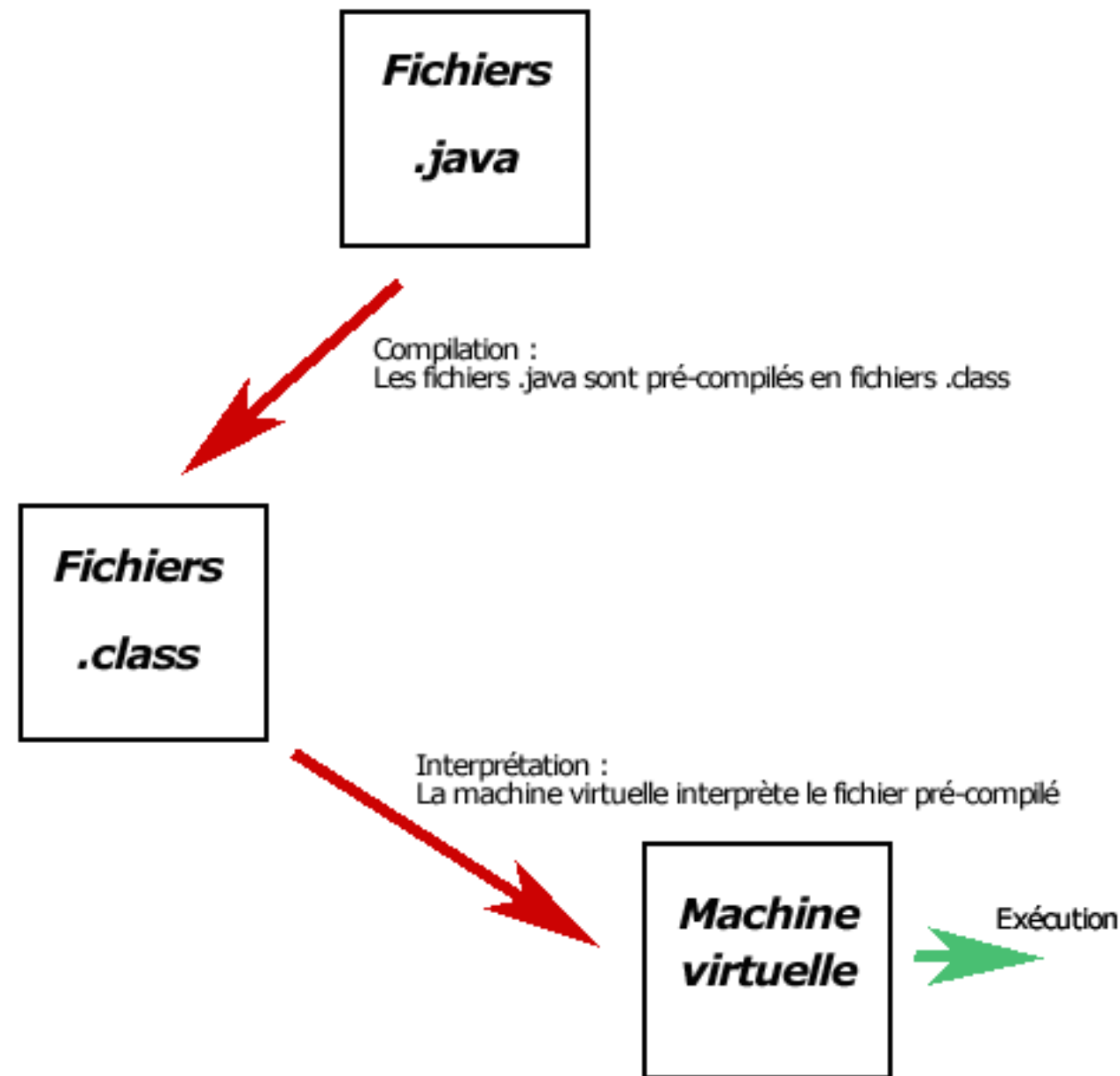
Les caractéristiques

Caractéristique	Signification
Langage interprété	Le code source est compilé en byte code et exécuté par la JVM
Portable	Le langage Java est indépendant de toute plateforme. Le code écrit peut s'exécuter n'importe où
Orienté objet	Java rassemble dans un même objet (classe) les propriétés et les fonctions qui s'y appliquent. Java est centré sur le paradigme orienté objet, utilisant des concepts tels que les classes , objets , héritage , polymorphisme , encapsulation et abstraction .
Fortement typé	Tous les attributs utilisés doivent avoir un type défini
Gestion de la mémoire	Java gère automatiquement l'allocation et la libération de la mémoire avec le Garbage Collector, ce qui réduit les risques de fuites de mémoire et d'erreurs liées à la gestion manuelle.
Multithreading	Permet l'utilisation de Threads qui sont des unités d'exécutions isolées. Le langage Java permet donc d'effectuer des traitements en parallèle

Les caractéristiques

Caractéristique	Signification
Sécurité	Java inclut des mécanismes pour éviter les failles courantes comme les dépassements de mémoire. Il dispose d'un modèle de sécurité robuste basé sur des classes et la gestion fine des droits d'accès.
Robustesse	Le système de gestion des erreurs de Java repose sur un mécanisme d'exceptions, ce qui permet d'écrire un code plus sûr et résistant aux bugs.
Extensibilité	Java est hautement extensible grâce à l'existence d'un écosystème riche de frameworks et de bibliothèques (Spring, Hibernate, Struts...).
Large communauté d'utilisateurs et Outils	Java bénéficie d'une large communauté de développeurs, d'un support étendu, et d'une gamme d'outils de développement tels qu'Eclipse, IntelliJ IDEA, et NetBeans. En raison de ces caractéristiques, Java est largement utilisé pour le développement d'applications d'entreprise, de systèmes embarqués, d'applications mobiles ...

Schéma d'exécution d'un code Java



Les principes de la programmation orientée objet

Encapsulation : C'est le fondement même de la programmation orientée objet. Il consiste à regrouper tout ce qui est lié à un objet au sein d'une classe. Cela implique donc d'intégrer chaque attribut et chaque méthode dans un seul emplacement.

Héritage : il consiste à intégrer les méthodes et les attributs d'une classe dans une autre classe. La classe de base est appelée classe mère et celle qui hérite est appelée la classe fille.

Polymorphisme : déclarer plusieurs fonctions avec le même nom, avec des attributs ou types de retour différents. Il permet de ne pas dupliquer de code, ce qui rend l'ensemble de la solution plus fluide et plus facile à maintenir.

Classe

Une classe est entité représentant un ensemble d'objets ayant la même structure et les mêmes comportements.

Syntaxe:

- public; private ou protected // portée de la classe
- abstract // classe impossible à instancier
- final // classe impossible à dériver
- class « nom de la classe »
- extends « nom de la classe mère »
- implements « nom d'interface » - { "corps de la classe" }

Exemple :

```
public class Article implements Serializable {  
    private static final long serialVersionUID = 7596696243276192383L;  
    private Long id;  
    private String code;  
    private String designation;  
    private String reference;  
    private Double prix;  
}
```

Package

Un **package** en Java est un « conteneur » utilisé pour regrouper les classes associées. Il représente un dossier dans un répertoire de fichiers. Les packages sont utilisés pour éviter les conflits de noms et pour écrire un code mieux maintenable.

Pour utiliser une classe ou un package de la bibliothèque, vous devez utiliser le mot-clé **import**

Exemple :

```
package net.ifri.model;
import java.io.Serializable;
import java.util.Date;

public class Article implements Serializable {
    /**
     * CODE
     */
}
```

Quelques bonnes pratiques de développement en JAVA

Le style de code

Bien indenter le code permet d'avoir un code clair et lisible. Cela permet aussi d'éviter les conflits avec les outils de versionning tels que SVN ou Git

```
/**
 * Mauvaise pratique
 */
public static File getPerformanceFile(String repName, String fileName) {
fileName = fileName + ".csv"; File fichier = new File(repName, fileName);
return fichier;}
```

```
/**
 * Bonne pratique
 */
public static File getPerformanceFile(String repName, String fileName) {
    fileName = fileName + ".csv";
    File fichier = new File(repName, fileName);
    return fichier;
}
```

Le style de code

Il est recommandé de mettre des accolades autour des structures de contrôle : if, for, while, try...

```
/**
 * Mauvaise pratique
 */
if (StringUtils.isNotBlank(bean.getQteStockMoyMensNat()))
    product.setQteStockMoyMensNat(Float.valueOf(bean.getQteStockMoyMensNat()));
else
    product.setQteStockMoyMensNat(null);
```

```
/**
 * Bonne pratique
 */
if (StringUtils.isNotBlank(bean.getQteStockMoyMensNat())) {
    product.setQteStockMoyMensNat(Float.valueOf(bean.getQteStockMoyMensNat()));
} else {
    product.setQteStockMoyMensNat(null);
}
```

Le style de code

Il est également préférable d'éviter les opérateurs ternaires qui sont difficiles à lire par certains développeurs. Cela rend le débogage du code fastidieux.

```
/**
 * Mauvaise pratique
 */
final Integer result = (isEnabled ()) ? getValidCode ( ) : (isBusy()) ? getBusyCode() : getErrorCode ( ) ;
```

```
/**
 * Bonne pratique
 */
Integer result = getErrorCode ();
if (isEnabled ()) {
    result = getValidCode ();
}

if (isBusy()) {
    result = getBusyCode();
}
```

Les métriques

Il s'agit d'indicateurs qui permettent de mesurer la qualité du logiciel.

Un bon code est clair, concis et facilement maintenable par d'autres développeurs.

Quelques indicateurs :

- **Le nombre de lignes de codes dans une classe**: La limite communément admise par classe est de 1000 lignes.
- **La longueur des méthodes**: La limite communément admise pour une méthode est de 100 lignes. Au delà, il faut un découpage en plusieurs méthodes et/ou classes de longueur raisonnable.
- **Le nombre de paramètres dans la signature d'une méthode**: La limite est de 7 par méthode. Plus une méthode comporte de paramètres, plus il est nécessaire de les regrouper dans un nouvel objet.

Les métriques

```
/**
 * Mauvaise pratique */
public void doSomething() { System.out.println("Hello world!");
// 200 copies omitted for brevity .}
```

```
/*
 * Bonne pratique ( java ) */
public void doSomething() {
    System.out.println("Hello world!");
    // 100 copies omitted for brevity
    doSubRoutine ( ) ;
}
public void doSubRoutine () {
    // 100 copies omitted for brevity
}
```


La complexité cyclomatique

Encore appelée mesure de McCabe, la complexité cyclomatique est une mesure représentant sous forme de graphe, le nombre de chemins qu'il est possible de suivre dans un algorithme.

La complexité cyclomatique s'incrémente à chaque instruction conditionnelle (if, else, case, for...) ajoutée à votre algorithme.

```
switch(x) {  
  case 1: return "a";  
  case 2: return "b";  
  case 3: return "c";  
  case 4: return "d";  
  case 5: return "e";  
  case 6: return "f";  
}  
return "other"; //complexité de 7
```

La complexité cyclomatique

Avantage: Un code simple à faible complexité cyclomatique est théoriquement plus facile à lire, à tester et à entretenir.

Inconvénient: Il ne faut pas rechercher systématiquement une complexité cyclomatique faible au risque de rendre le code moins lisible.

```
switch(x) {  
  case 1: return "a";  
  case 2: return "b";  
  case 3: return "c";  
  case 4: return "d";  
  case 5: return "e";  
  case 6: return "f";  
}  
return "other";//complexité de 7
```

```
var retValues = ["a", "b", "c", "d", "e", "f"];  
  
if (x > 0 && x <= 6) {  
  return retValues[x - 1];  
}  
return "other";//complexité de 3  
  
Code pas compréhensible cependant
```

Les métriques

Des outils OpenSource permettent de calculer les métriques de votre projet.
Exemple: SonarQube, Metrics, Checkstyle, EclEmma.

Focus sur le plugin eclipse « **EclEmma** » et présentation de SonarQube

Mise en place en TP

Le Pair Programming



Les design pattern en JAVA

Les design patterns

Les design patterns sont apparus dans le but de faciliter la maintenance et la réutilisabilité du code.

On est parti du constat que le nombre d'applications développées avec des technologies orientées objet augmente. Alors l'idée de réutiliser des techniques pour solutionner des problèmes courants a abouti au recensement d'un certain nombre de modèles connus sous le nom de motifs de conception (design patterns).

Ces modèles sont définis pour être utilisés avec un maximum de langages de développement orientés objet.

Origine des design patterns

L'origine des Design Patterns remonte au début des années 1970 avec les travaux de l'architecte Christopher Alexander qui remarqua que la phase de conception en architecture fait apparaître des problèmes récurrents.

Il chercha alors à résoudre l'ensemble de ces problèmes liés à des contraintes interdépendantes (solidité de la structure, étanchéité...).

Pour cela il établit un langage de 253 patterns, qui couvrent tous les aspects de la construction (comme par exemple la façon de concevoir une charpente).

Les Design Patterns trouvent leur origine donc dans le domaine du bâtiment et de la construction.

Origine des design patterns

A la suite dans les années 90, l'idée de Christopher Alexander va être reprise et étendue au domaine de la conception des logiciels.

Le concept de Design Pattern est développé dans un ouvrage publié en 1995 par une équipe composée de Erich Gamma, Richard Helm, Ralph Johnson et John Vlissides. L'ouvrage s'intitule '**Design Patterns : Elements of Reusable Object-Oriented Software**'.

Cet ouvrage présente 23 Design Patterns qui font aujourd'hui référence dans le monde de l'informatique.

Les design patterns en Java

En Java, les design patterns sont des solutions éprouvées à des problèmes récurrents dans le développement logiciel.

Ils aident à concevoir des applications robustes, maintenables et évolutives.

Les design patterns sont généralement classés en trois catégories principales : **créationnels**, **structurels** et **comportementaux**.

Les catégories de design pattern

Il existe 3 grandes catégories ces modèles de conception :

- **les modèles créationnels (creational patterns)** : ils permettent d'instancier et de configurer des classes et des objets. Ils concernent la manière de créer des objets tout en masquant la logique complexe de création.
- **les modèles structurels (structural patterns)** : ils permettent d'organiser les classes d'une application. Ces modèles traitent de la composition des classes et des objets pour former des structures plus grandes.
- **les modèles comportementaux (behavioral patterns)** : ils permettent d'organiser les objets pour qu'ils collaborent entre eux. Ils s'intéressent à l'interaction entre les objets et à la manière de répartir les responsabilités.

Les modèles de création

Dans cette catégorie on distingue généralement 5 modèles :

Nom	Rôle
Singleton	Design pattern garantissant qu'une classe qui ne pourra avoir qu'une seule instance pendant l'exécution du programme et fournit un point d'accès global à celle-ci.
Factory	Design pattern définit une méthode pour créer des objets sans spécifier leur classe concrète.
Abstract Factory	Fournir une interface unique pour instancier des objets d'une même famille sans avoir à connaître les classes à instancier
Builder	Modèle utilisé pour la création d'objets complexes dont les différentes parties doivent être créées suivant un certain ordre ou algorithme spécifique.
Prototype	Création d'objet à partir d'un prototype

Le Singleton

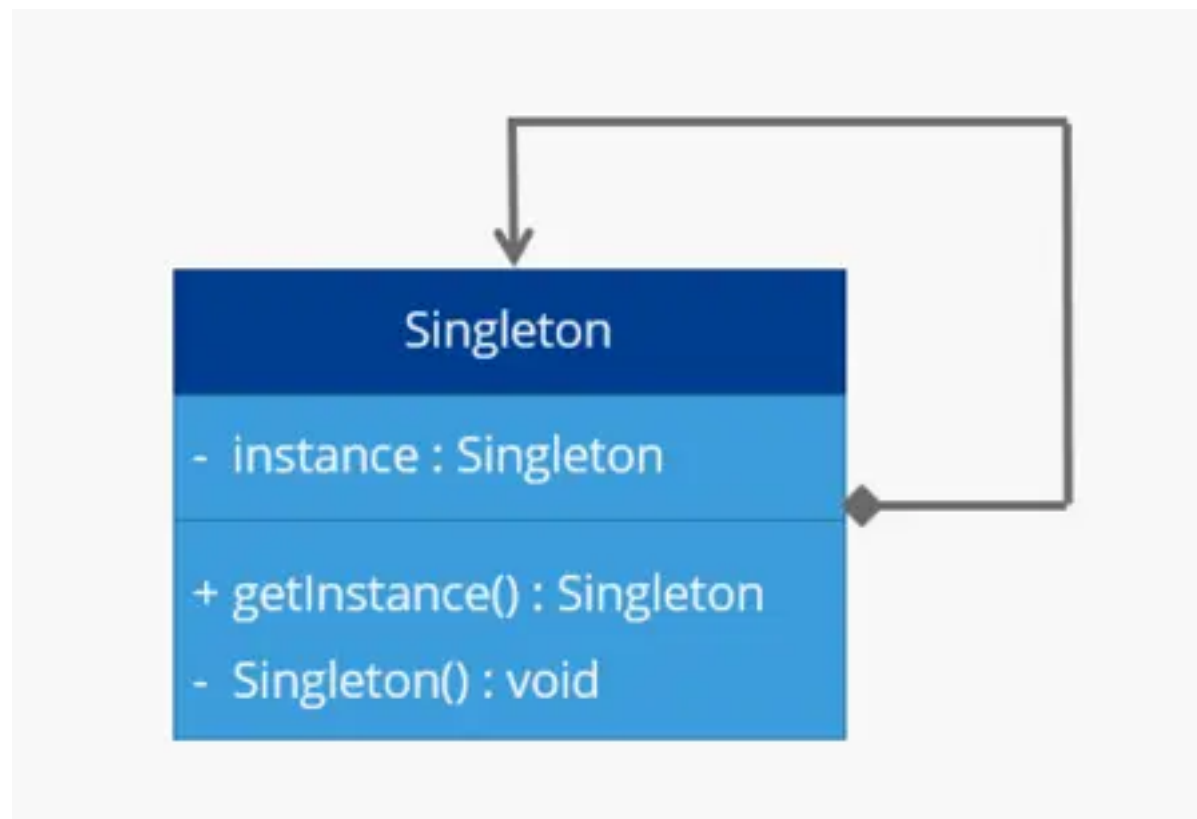
Un singleton est une classe qui ne peut avoir qu'une seule et unique instance dans un programme.

Un singleton est souvent pratique lorsqu'on souhaite manipuler des classes sans état qui effectuent toujours les mêmes traitements.

Ainsi une classe qui ne comporte que des méthodes statiques doit être transformée en un singleton.

Pour éviter toute instanciation extérieure à cette classe, il est nécessaire de rajouter un constructeur privé dans la classe Singleton.

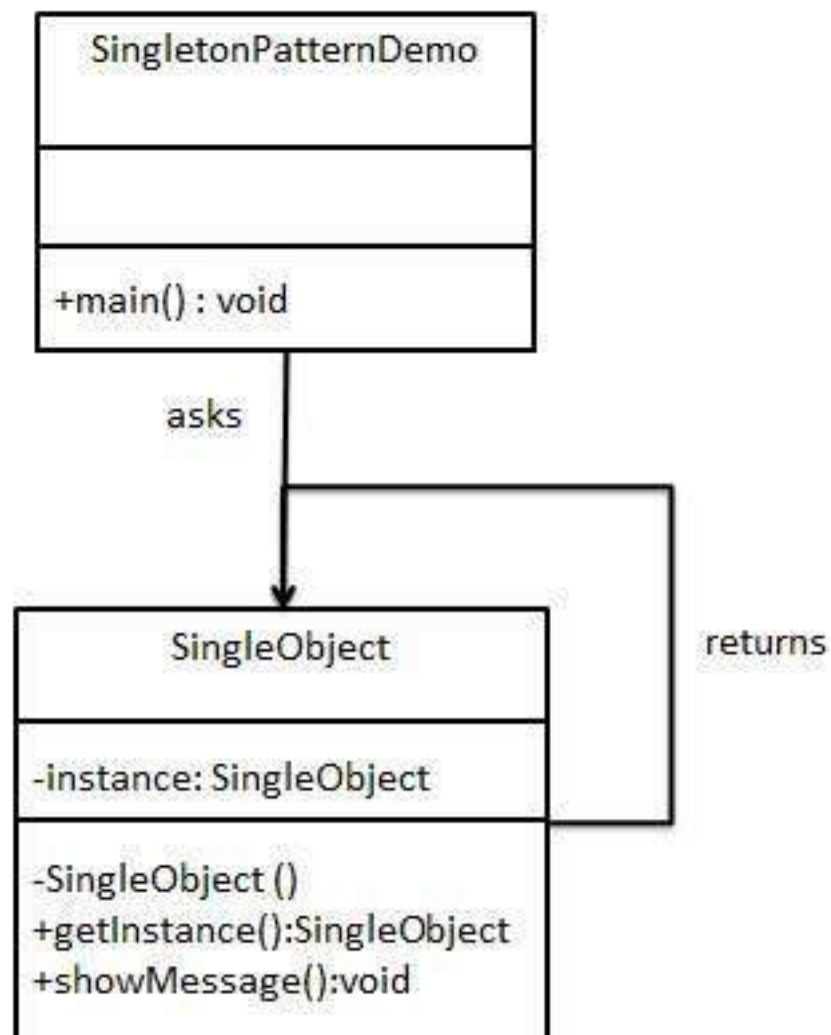
Représentation UML du design pattern Singleton



Le Singleton

```
public class Singleton {  
    private static Singleton instance;  
    private Singleton() {}  
  
    public static Singleton getInstance() {  
        if (instance == null) {  
            instance = new Singleton();  
        }  
        return instance;  
    }  
}
```

Le Singleton



Le pattern Builder

Le pattern Builder permet de construire des objets complexes à partir d'objets simples en se basant sur une approche pas-à-pas.

Le pattern Builder est utilisé pour la création d'objets complexes dont les différentes parties doivent être créées suivant un certain ordre ou algorithme spécifique. Une classe externe contrôle l'algorithme de construction.

Il permet ainsi de dissocier la construction d'un objet de sa représentation, afin que le même processus de construction ait la possibilité de créer des représentations différentes.

Le pattern Builder

Ce design pattern sera préconisé dans les situations où au moins :

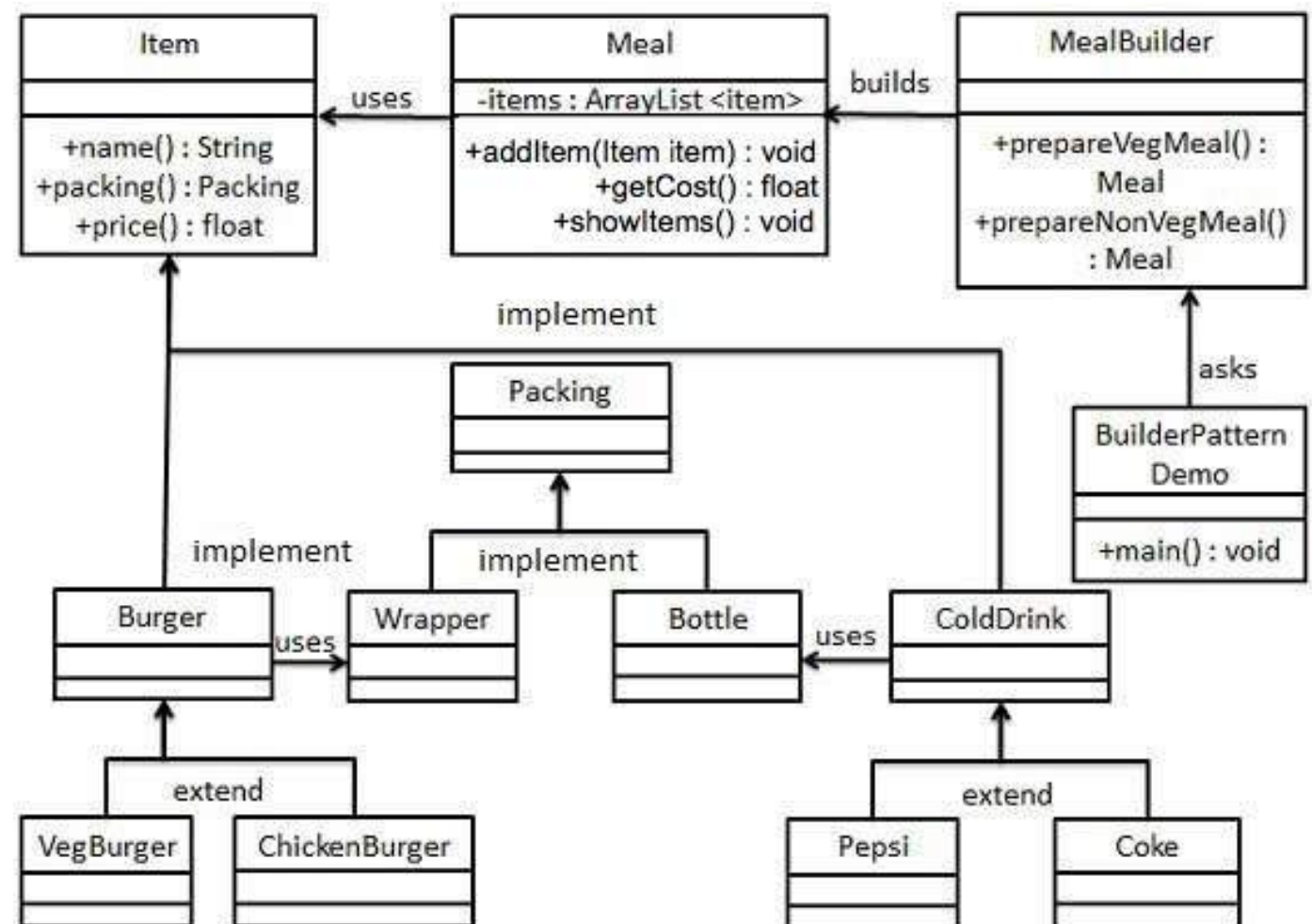
- l'objet final est imposant, et sa création complexe ;
- beaucoup d'arguments doivent être passés à la construction de l'objet, afin d'avoir un design lisible ;
- certains de ces arguments sont optionnels (par ex. : un bateau n'a pas forcément de capitaine, mais toujours une taille), ou ont plusieurs variations (la taille peut par exemple être passée en mètres ou en pieds).

Le pattern Builder

TP d'implémentation : Nous avons considéré le cas d'un restaurant de restauration rapide où un repas typique pourrait être un hamburger et une boisson fraîche. Le burger peut être un burger végétarien ou un burger au poulet et sera emballé dans un emballage. La boisson froide peut être un coca ou un pepsi et sera emballée dans une bouteille.

Nous allons créer une interface `Item` représentant des produits alimentaires tels que des hamburgers et des boissons froides et des classes concrètes implémentant l'interface `Item` et une interface `Packing` représentant l'emballage de produits alimentaires et des classes concrètes implémentant l'interface `Packing` car le hamburger serait emballé dans un emballage et une boisson froide, serait emballé en bouteille.

Nous créons ensuite une classe `Meal` ayant `ArrayList` of `Item` et un `MealBuilder` pour créer différents types d'objets `Meal` en combinant `Item`. `BuilderPatternDemo`, notre classe de démonstration utilisera `MealBuilder` pour créer un repas.



Le pattern Builder



Le pattern Builder

Etape 1: Création des interfaces Packing et Item

```
public interface Packing {  
    public String pack();  
}
```

```
public interface Item {  
    public String name();  
    public Packing packing();  
    public float price();  
}
```

Etape 2: Création des classes Wrapper et Bottle implémentant l'interface Packing

```
public class Wrapper implements Packing {  
    @Override  
    public String pack() {  
        return "Wrapper";  
    }  
}
```

```
public class Bottle implements Packing {  
    @Override  
    public String pack() {  
        return "Bottle";  
    }  
}
```

Le pattern Builder

Etape 3: Création des classes Burger et ColdDrink implémentant l'interface Item

```
public abstract class Burger implements Item {  
  
    @Override  
    public Packing packing() {  
        return new Wrapper();  
    }  
  
    @Override  
    public abstract float price();  
}
```

```
public abstract class ColdDrink implements Item {  
  
    @Override  
    public Packing packing() {  
        return new Bottle();  
    }  
  
    @Override  
    public abstract float price();  
}
```

Le pattern Builder

Etape 4: Création des classes VegBurger, ChickenBurger, Pepsi et Coca qui étendent Burger et ColdDrink

```
public class VegBurger extends Burger {  
  
    @Override  
    public float price() {  
        return 25.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Veg Burger";  
    }  
  
}
```

```
public class ChickenBurger extends Burger {  
  
    @Override  
    public float price() {  
        return 50.5f;  
    }  
  
    @Override  
    public String name() {  
        return "Chicken Burger";  
    }  
  
}
```

Le pattern Builder

Etape 4: Création des classes VegBurger, ChickenBurger, Pepsi et Coca qui étendent Burger et ColdDrink

```
public class Coke extends ColdDrink {  
  
    @Override  
    public float price() {  
        return 30.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Coke";  
    }  
  
}
```

```
public class Pepsi extends ColdDrink {  
  
    @Override  
    public float price() {  
        return 35.0f;  
    }  
  
    @Override  
    public String name() {  
        return "Pepsi";  
    }  
  
}
```


Le pattern Builder

Etape 6: Création de la classe Meal

```
import java.util.ArrayList;
import java.util.List;

public class Meal {
    private List<Item> items = new ArrayList<Item>();

    public void addItem(Item item) {
        items.add(item);
    }

    public float getCost() {
        float cost = 0.0f;
        for (Item item : items) {
            cost += item.price();
        }
        return cost;
    }

    public void showItems() {
        for (Item item : items) {
            System.out.println("Item : " + item.name() + ", Packing : " + item.packing().pack() + ", Price : " + item.price());
        }
    }
}
```


Le pattern Builder

Etape 6: Création du builder MealBuilder qui est le pattern chargé de créer les objets Meal

```
public class MealBuilder {  
    public Meal prepareVegMeal () {  
        Meal meal = new Meal();  
        meal.addItem(new VegBurger());  
        meal.addItem(new Coke());  
        return meal;  
    }  
    public Meal prepareNonVegMeal () {  
        Meal meal = new Meal();  
        meal.addItem(new ChickenBurger());  
        meal.addItem(new Pepsi());  
        return meal;  
    }  
}
```

Etape 7: La classe BuildPatternDemo permettant de lancer le programme

```
public class BuilderPatternDemo {  
    public static void main(String[] args) {  
        MealBuilder mealBuilder = new MealBuilder();  
        Meal vegMeal = mealBuilder.prepareVegMeal();  
        System.out.println("Vegetarian Meal");  
        vegMeal.showItems();  
        System.out.println("Total Cost: " + vegMeal.getCost());  
  
        Meal nonVegMeal = mealBuilder.prepareNonVegMeal();  
        System.out.println("Chicken Meal");  
        nonVegMeal.showItems();  
        System.out.println("Total Cost: " + nonVegMeal.getCost());  
    }  
}
```

Le pattern Factory

Le pattern factory (ou fabrique de création) est l'un des designs pattern les plus utilisés en Java.

Son rôle est de construire des objets sans exposer au client l'origine de ces objets. Les objets ainsi créés sont appelés en utilisant une interface commune.

La fabrique permet de créer un objet dont le type dépend du contexte : cet objet fait partie d'un ensemble de sous-classes. L'objet retourné par la fabrique est donc toujours du type de la classe mère mais grâce au polymorphisme les traitements exécutés sont ceux de l'instance créée.

Ce motif de conception est utilisé lorsqu'à l'exécution il est nécessaire de déterminer dynamiquement quel objet d'un ensemble de sous-classes doit être instancié.

Le pattern Factory

Le pattern factory Il est utilisable lorsque :

- Le client ne peut déterminer le type d'objet à créer qu'à l'exécution
- Il y a une volonté de centraliser la création des objets

L'utilisation d'une fabrique permet de rendre l'instanciation d'objets plus flexible que l'utilisation de l'opérateur d'instanciation new.

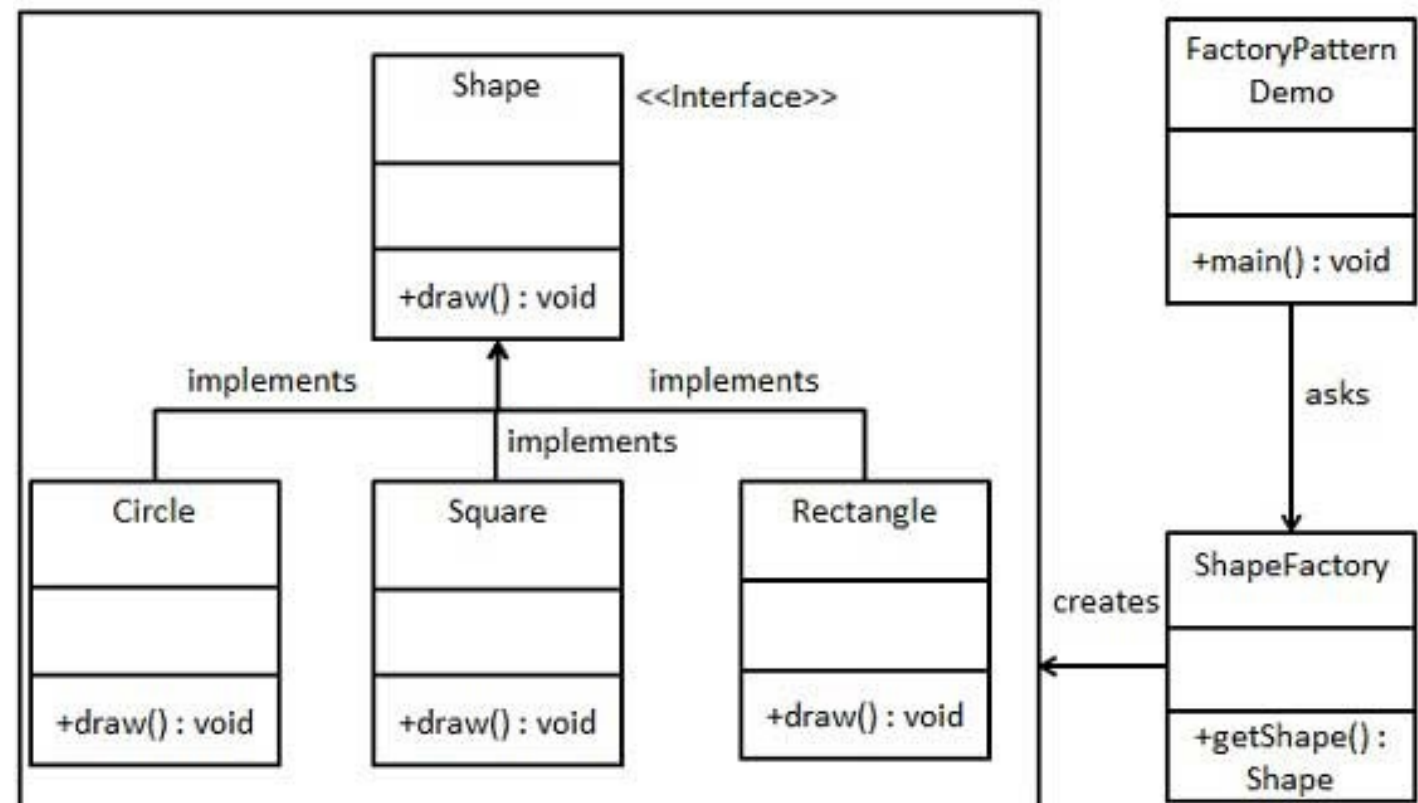
Ce design pattern peut être implémenté en déclarant une fabrique dont la méthode de création de l'objet attend les données nécessaires pour déterminer le type de l'objet à instancier

Le Factory

```
public interface Product {  
    void use();  
}  
  
public class ConcreteProduct implements Product {  
    public void use() {  
        System.out.println("Product used");  
    }  
}  
  
public class Factory {  
    public static Product createProduct() {  
        return new ConcreteProduct();  
    }  
}
```

Le pattern Factory

TP : Using the following UML Diagram, we're going to create a Shape interface and concrete classes implementing the Shape interface. A factory class ShapeFactory is defined as a next step. FactoryPatternDemo, our demo class will use ShapeFactory to get a Shape object. It will pass information (CIRCLE / RECTANGLE / SQUARE) to ShapeFactory to get the type of object it needs.



Le pattern Factory

L'interface Shape.java

```
public interface Shape {  
    public void draw();  
}
```

La classe Shape.java

```
public class Circle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Circle object => draw() method.");  
    }  
}
```

La classe Square.java

```
public class Square implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Square object => draw() method.");  
    }  
}
```

Le pattern Factory

La classe Rectangle.java

```
public class Rectangle implements Shape {  
    @Override  
    public void draw() {  
        System.out.println("Inside Rectangle object => draw() method.");  
    }  
}
```

La factory ShapeFactory.java

```
public class ShapeFactory {  
    public Shape getShape(String type) {  
        if(("CIRCLE").equalsIgnoreCase(type)) {  
            return new Circle();  
        }  
        if (("RECTANGLE").equalsIgnoreCase(type)) {  
            return new Rectangle();  
        }  
        if (("SQUARE").equalsIgnoreCase(type)) {  
            return new Square();  
        }  
        return null;  
    }  
}
```


Le pattern Factory

La classe principale **FactoryPatternDemo.java**

```
public class FactoryPatternDemo {  
    public static void main(String[] args) {  
        ShapeFactory shapeFactory = new ShapeFactory();  
  
        Shape shape1 = shapeFactory.getShape("CIRCLE");  
        shape1.draw();  
  
        Shape shape2 = shapeFactory.getShape("RECTANGLE");  
        shape2.draw();  
  
        Shape shape3 = shapeFactory.getShape("SQUARE");  
        shape3.draw();  
    }  
}
```

En exécutant la classe **FactoryPatternDemo**, la sortie console affichera :

```
Inside Circle object => draw() method.  
Inside Rectangle object => draw() method.  
Inside Square object => draw() method.
```


Les règles de codage

Le camelCase:

Méthode la plus utilisée pour nommer en Java. Elle consiste à organiser le nommage de la manière suivante: chaque première lettre d'un mot prend une majuscule, tous les mots sont collés les uns aux autres. Exemple : `AnneeScolaire.java`, `AnneeService.java`...

Les packages:

Ils doivent être nommés avec un nom de domaine inversé auquel on ajoute le nom du projet. Exemple: `bj.ifri.sendmail`. Ce système de nommage permet entre autres d'éviter les conflits entre les classes portant le nom.

Les règles de codage

Les classes et interfaces

Les classes et les interfaces sont nommées selon la méthode camelCase mais commencent par une majuscule. Il faut utiliser des mots clairs qui décrivent bien la classe et sa fonction. Exemple: Book.java

Les méthodes

Les méthodes sont nommées selon la méthode CamelCase et comportent généralement un verbe. Exemple : `crediterCompte()`, `acheterVoiture()`...

Les règles de codage

Les variables

Une variable doit avoir un nom clair et concis. Cela permet d'identifier facilement sa responsabilité. Les noms de variable à un caractère sont déconseillés.

Les constantes

Les constantes sont des variables déclarées « static final ». Leur contenu ne change pas durant toute l'exécution du programme. Les constantes sont nommées en majuscule. Exemple: SEUIL_DECOUVERT, HAUTEUR_MIN.

Gestion des dépendances en Java avec Maven

Définitions et utilité de Maven

Maven est un outil de construction de projets open source, développé par Apache. Il permet de faciliter et d'automatiser certaines tâches de la gestion d'un projet Java :

- Gestion des dépendances : librairies (Jar) bibliothèques nécessaires au projet
- compilation du code source
- Exécution des tests unitaires
- Packaging du projet sous forme de Jar ou War pour déploiement (génération une application déployable à partir d'un code source)
- Génération des documentations (Javadoc) du projet

Installation et configuration

Maven est un outil développé en langage Java et qui nécessite donc que la JVM Java soit donc installée sur la machine.

Généralement, surtout sur un ordinateur qui n'est pas connecté sur un réseau d'entreprise disposant d'un gestionnaire de dépôts, une connexion à internet est nécessaire pour permettre le téléchargement des plugins requis et des dépendances.

Installation et configuration

L'installation de Maven se fait en suivant les étapes suivantes :

- Télécharger l'archive sur le site <http://maven.apache.org/download.html>
- Décompresser l'archive dans un répertoire sur la machine
- Créer la variable d'environnement **M2_HOME** qui pointe sur le répertoire contenant Maven
- Ajouter le chemin **M2_HOME/bin** à la variable PATH du système
- Vérifier l'installation en lançant la commande **mvn -version** dans le terminal (pour les machines Linux ou Mac) ou dans l'invite de commande cmd (pour les machines windows)

Unix-based operating systems (Linux, Solaris and Mac OS X) : export PATH=/usr/local/apache-maven-3.x.y/bin:\$PATH

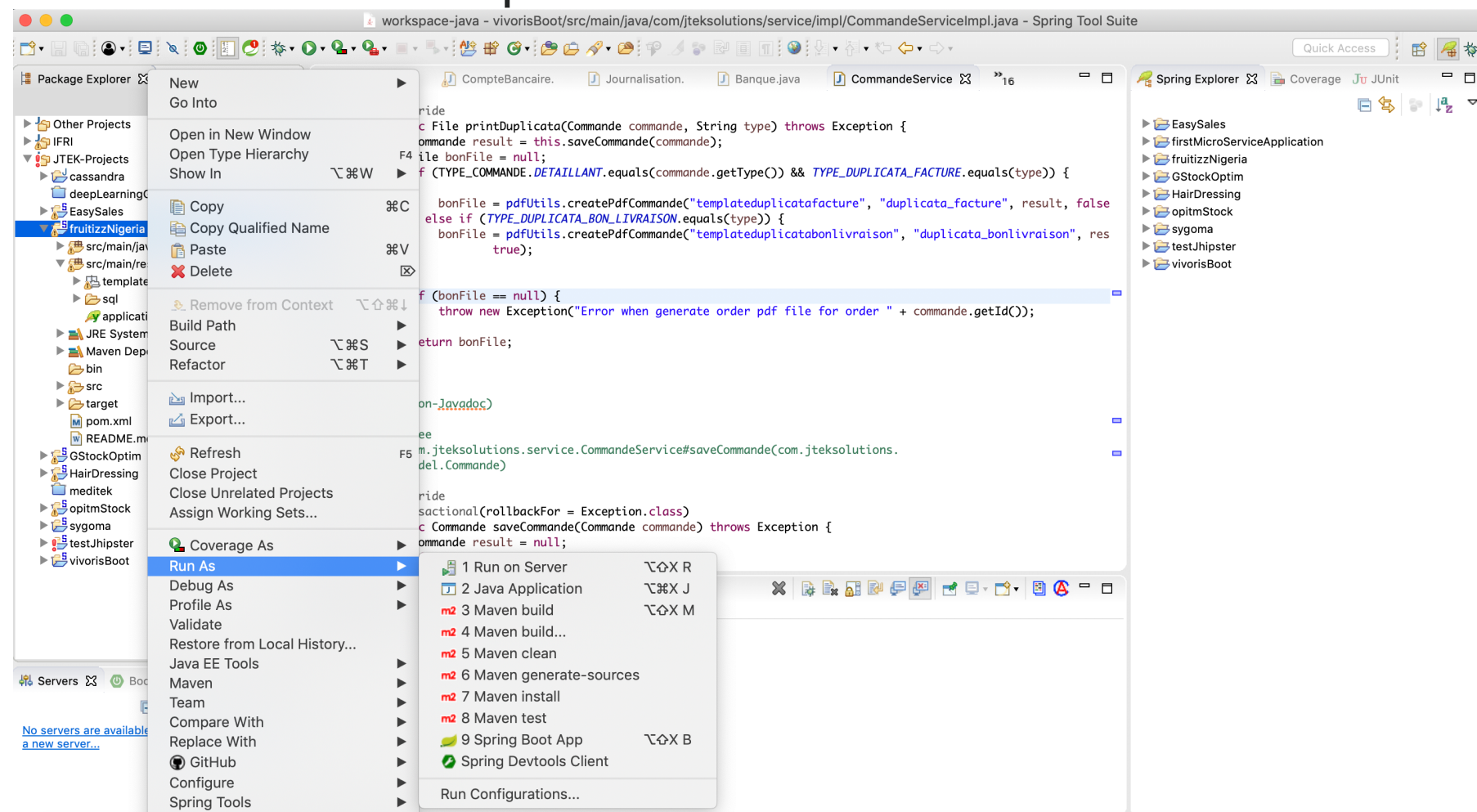
Windows : set PATH="c:\program files\apache-maven-3.x.y\bin";%PATH%

- *mvn archetype:generate -DgroupId=bj.ifri.tpmaven -DartifactId=tpmaven -DarchetypeArtifactId=maven-archetype-quickstart -DarchetypeVersion=1.5 -DinteractiveMode=false*

Installation et configuration

Spring Tool Suite (STS) est un IDE étendu pour Eclipse. Il se spécialise dans le développement des applications Spring.

Cet IDE possède une version embarquée de Maven. Nous allons utiliser l'IDE Eclipse Spring Tool Suite.



Structure générale d'un projet Maven

Par convention, l'arborescence d'un projet Maven est toujours la même par défaut et est imposée par Maven. Cette convention a été mise en place pour permettre aux développeurs de se familiariser avec la structure des projets car elle est toujours la même.

```
./pom.xml  
./src  
./src/main  
./src/main/java  
./src/main/resources  
./src/test  
./src/test/java  
./src/test/resources
```

Structure générale d'un projet Maven

Répertoires / fichiers	Contenu du répertoire
pom.xml	le fichier POM (Project Object Model) de description du projet
src/main/java	Le code source du projet (les classes Java)
src/main/resources	les fichiers de ressources du projet (fichiers de configuration, images, ...)
src/test/java	le code source des tests unitaires
src/test/resources	les fichiers de ressources pour les tests

Structure d'un fichier pom.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.ifri</groupId>
  <artifactId>javaavance</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>javaavance</name>
  <description>Project description</description>

  <properties>
    <java.version>1.10</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.postgresql</groupId>
      <artifactId>postgresql</artifactId>
      <version>1.0.0</version>
      <scope>compile</scope>
    </dependency>
  </dependencies>

</project>
```

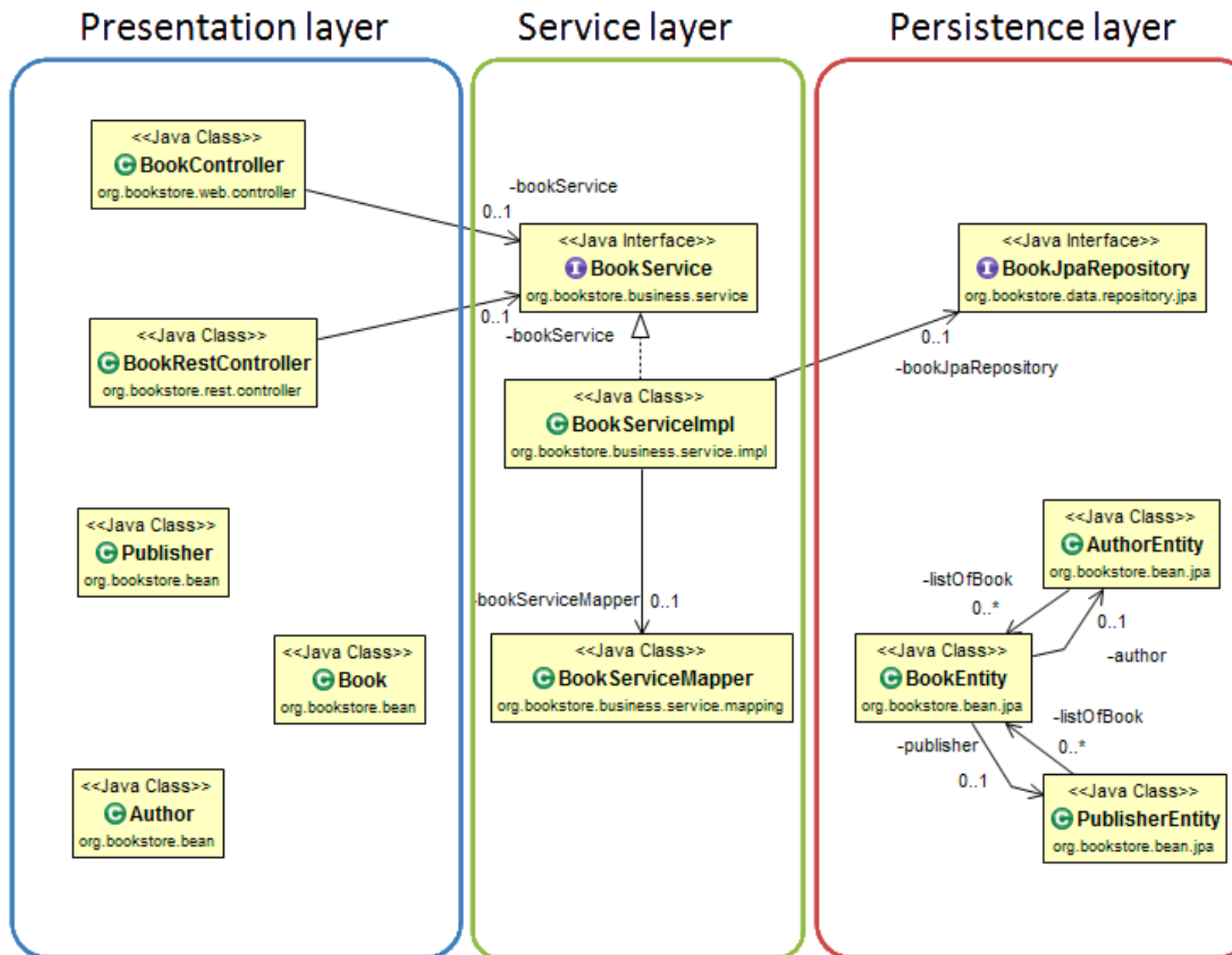
Le framework Spring

Le framework Spring

Le framework Spring est une boîte à outils très riche permettant de créer facilement des applications JEE. Le framework Spring est organisé en modules :

- L'injection de dépendance ou IOC (Injection Of Dependency),
- Gestion des instances de classes
- Programmation orientée Aspect (AOP) : séparation du code technique du code métier.
Similaire au concept des design patterns,
- Modèle MVC (Modèle-Vue-Contrôleur) et outils pour les applications WEB,
- Outils pour la DAO (JDBC),
- Outils pour les ORM (Hibernate, iBatis, JPA ...),
- Outils pour les applications JEE (JMX, JMA, JCA, EJB, ...),

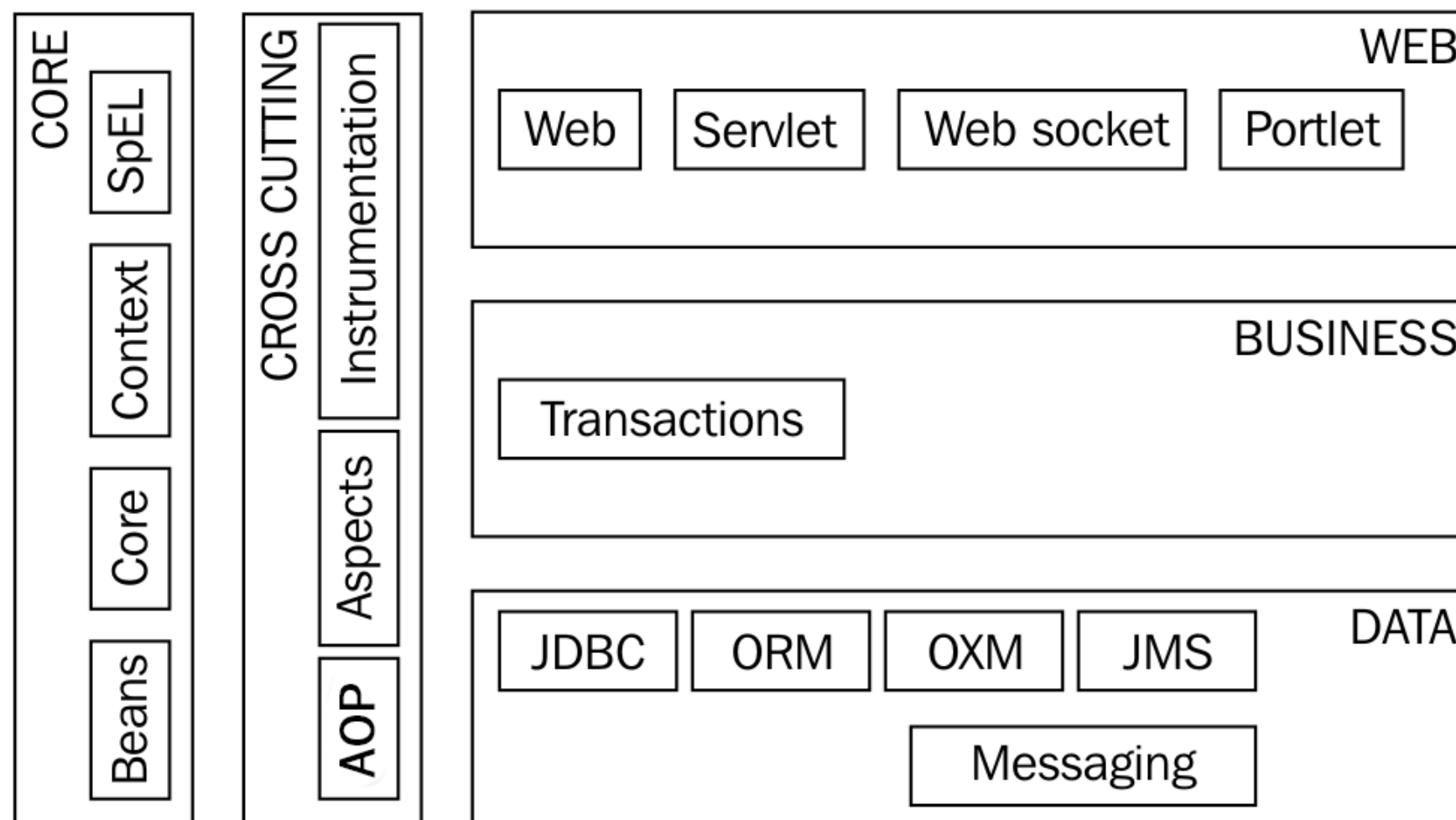
Le framework Spring



Le framework Spring

<https://gayerie.dev/docs/spring/spring/introduction.html#les-modules-spring-framework>

Les modules Spring sont :



Présentation de SpringBoot

Présentation de SpringBoot

Spring est un framework bien connu des développeurs Java pour les nombreuses fonctionnalités qu'il apporte sur les aspects web, sécurité, batch ou encore accès aux données dans le cadre du développement d'une application. Mais il est aussi malheureusement connu pour sa configuration qui peut s'avérer complexe et fastidieuse. Il n'était pas rare de passer plusieurs jours sur la configuration d'un projet Spring notamment pour des personnes novices dans l'utilisation du framework. Pour corriger cette anomalie, les équipes de Spring ont décidé de travailler sur un projet permettant de faciliter le développement d'application Spring pour les développeurs. C'est ainsi qu'est né Spring Boot.

Présentation de SpringBoot

Spring Boot est un framework Spring qui a pour but de faciliter la configuration d'un projet Spring et de réduire le temps alloué au démarrage d'un projet. Pour cela Spring Boot propose 2 fonctionnalités principales qui sont l'auto-configuration et les starters.

L'auto-configuration : Cette fonctionnalité est la plus importante de Spring Boot. Elle permet de configurer automatiquement votre application à partir des *jar* trouvés dans votre Classpath.

L'annotation `@EnableAutoConfiguration` permet d'activer l'auto-configuration sur un projet SpringBoot

Présentation de SpringBoot

Les starters : Les starters viennent compléter l'auto-configuration et font gagner énormément de temps, notamment lorsqu'on commence le développement d'une application. Un starter va apporter à votre projet un ensemble de dépendances, communément utilisées pour un type de projet donné. Ceci va vous permettre de créer un "squelette" prêt à l'emploi très rapidement.

Configuration d'un projet SpringBoot

Voir TP

<https://start.spring.io/>

Communication avec une base de données en Java : cas de Spring JPA

Introduction à Spring Data JPA

Spring Data est un robuste framework de l'écosystème de Spring qui se compose de nombreux modules offrant des solutions de communication avec les systèmes de gestion de bases de données comme Mysql, Postgresql, Oracle, MongoDB, Neo4j...

Spring Data JPA met en place une surcouche d'accès à JPA, et fournit donc ainsi un ensemble cohérent de fonctionnalités avancées sur lesquels s'appuyer pour bâtir et consolider les applications d'entreprise en Java.

Fonctionnalités de Spring Data JPA

- Aide à la construction de Repository Spring en automatisant l'instanciation des services DAO (Data Access Object) qui sont annotés `@Repository` en Spring. Ce mécanisme se base sur la simple définition d'interface en Java.
- Pagination et tri : Spring Data JPA permet de résoudre simplement le problème classique de chargement de données paginées (ex : charger la 3ème page d'un volume de 15 éléments). De même, Spring Data JPA offre des primitives afin de gérer correctement et simplement les mécanismes de requêtes utilisant le tri sur des propriétés.

Fonctionnalités de Spring Data JPA

- Spring Data met en place des Query, générant automatiquement des implémentations de Finder basées sur les signatures de méthodes et une nomenclature de propriétés.

Exemple : On définit une interface `ProductRepository` dans laquelle on définit les méthodes suivante :

- **findByPrice** : méthode de recherche sur la propriété `Product.price`
- **findByNomLike** : méthode de recherche sur la propriété `Product.designation` tout en opérant avec une politique de recherche SQL Like.

ANNEXES

- Les métriques: baptiste-wicht.developpez.com/tutoriels/eclipse/metrics
- Le nombre cyclomatique: https://fr.wikipedia.org/wiki/Nombre_cyclomatique
- Les designs patterns: http://www.tutorialspoint.com/design_pattern/factory_pattern.htm
- Les threads : http://www.tutorialspoint.com/java/java_multithreading.htm
- Conventions de nommage en java : <https://openclassrooms.com/forum/sujet/conventions-de-codage-en-java-64171>
- Présentation de Springboot : <http://www.neo-soft-solutions.fr/spring-boot-kesako/>
- Introduction aux design patterns : <https://design-patterns.fr/introduction-aux-design-patterns#:~:text=Un%20Design%20Pattern%20est%20une,probl%C3%A8me%20d'architecture%20de%20logiciel>
- <https://gayerie.dev/docs/spring/spring/introduction.html>
- https://gayerie.dev/docs/spring/spring/spring_data_jpa.html
- https://gayerie.dev/docs/spring/spring/spring_mvc_intro.html
- <https://start.spring.io/>